

Two steps are needed to find an exact match in the grid. First, the a, b coordinates are transformed into the row, column of the corresponding cell by calling `__getCell()`, as shown in [Listing 12-16](#). If a point with a, b coordinates exists in the `Grid`, it must be found in that cell. The Python expression `__cells.get(cell, None)` looks to see whether the `__cells` dictionary has a value stored for the `cell` key and returns it, if so. It returns the second argument, `None`, when the key is not in the dictionary. The result is put in the `points` variable.

Next, if the cell contains any points, the `findExact()` method loops through each of them. If the exact match is found, then the method returns the associated data. If no match is found, then the loop ends and the method just returns `None`.

Listing 12-16 *Finding an Exact Match in a Grid Object*

```
class Grid(object):
...

    # Find the data associated with exact match of coordinates
    def findExact(self, a, b):
        cell = self.__getCell(a, b) # which cell contains a,b
        points = self.__cells.get(cell, None) # get list of points
        if points: # there are points for the cell
            for p in points: # check each point and seek a match
                if p[0] == a and p[1] == b:
                    return p[2] # return the data for exact match
        return None
```

Big O and Practical Considerations

What is the Big O for `findExact()`? There is a theoretical answer and then a more practical answer that suggests that one must also consider a pragmatic, engineering perspective.

From a theoretical perspective, both `findExact()` and `insert()` are $O(N)$. Let's review why. The first step in both methods transforms the a, b coordinates into a row, column pair that specifies the cell at which the a, b point might be stored. This can be done in $O(1)$ time. Now that you know which cell might contain the search point, the code loops through all the points stored in the cell's list looking for an exact match. The Big O of this step depends on how

the length of each list grows as the overall number of points N grows. Suppose that the grid had five rows and five columns, and suppose that the points are uniformly spread across the 25 grid cells. Then you can expect that there would be about $N/25$ points stored in each cell. Because you throw away the constant factor in analyzing complexity, looping through $N/25$ points to find a match takes $O(N)$ time.

This scenario seems somewhat extreme. For example, if you have 1 million points, a five-by-five grid isn't appropriate. By taking a pragmatic approach, if the grid instead had 500 by 500 cells, then the million points would be spread out to about 4 points per each of the 250,000 cells, which surely seems like an $O(1)$ operation to do an exact find! But from a theoretical perspective, the algorithm remains $O(N)$ because Big O describes how the computation time increases *as N grows larger*. If you were to keep the grid fixed at 500×500 cells, and then allow N to grow from 1 million points to 10 million, you would expect the computation time to likewise increase linearly by a factor of 10.

From a practical perspective, if you know that you have a fixed, or even very slowly growing, number of points, then it can be appropriate, if you have enough memory, to engineer your grid to contain enough cells so that each cell contains a tiny number of points, resulting in effectively constant time to process the cell's points. This approach offers the possibility of a classic engineering trade-off between time efficiency and space or memory efficiency for a specific scenario of known size. In the example of a 500×500 grid storing a fixed amount of 1 million points, you have effectively used $O(N)$ memory for cells and lists to obtain an *approximation* of $O(1)$ time performance in both `findExact()` and `insert()`. Even with that engineered solution, however, it is important to remember that the exact find operation is still $O(N)$ for a fixed grid size as the number of points N grows.

In the next section, we consider quadtrees, which have a much more attractive $O(\log N)$ time complexity to find an exact match. Does that mean you will always want to use quadtrees? Not necessarily.

When coding actual applications, you often know the practical range of amounts of data points you may have to process. Toward the end of this chapter, we take a practical look at how to choose among spatial data structure solutions by timing them on different sized test data. As you will see, the most theoretically attractive solution doesn't always result in the shortest run time.

Deleting and Traversing

We leave it as a Programming Project for you to implement a method to delete from the `Grid` a point specified by a pair of coordinates. The approach should be similar to that used by `findExact()`, except that after the matching point is identified, it is removed from the cell's list. Just like `findExact()`, the Big O for `Grid`'s `delete` is also $O(N)$.

Traversing the grid is straightforward as long as deletion correctly removes points from the cells and empty cells are skipped. Because the `Grid` is implemented as grid cells containing simple lists of tuples, you can implement the `traverse()` method as a generator using two nested loops. [Listing 12-17](#) shows the implementation.

Listing 12-17 *The `traverse()` Method of `Grid`*

```
class Grid(object):  
  
...  
    def traverse(self):  
        for cell in self.__cells:           # for each cell in the grid  
            for p in self.__cells[cell]:    # for each point in the cell  
                yield p                     # yield the point
```

The `traverse()` generator loops through each of the nonempty cells, getting the cell key from the `_cells` hash table. That key is the row, column tuple of the cell. The inner loop steps through all the points stored in that cell, yielding each point. The points are tuples that contain the *a*, *b* coordinates and the associated data. The caller can use tuple assignment in the `for` loop to assign the values to three variables or just assign the returned tuple to a single loop variable. This method has $O(N)$ complexity.

Finding the Nearest Match

Sometimes it is fast and easy to find the nearest match to a query point. The method can start just like `findExact()`, the coordinates of the query point are converted into a grid cell's row, column coordinates, and then all the points stored in that grid cell are checked to identify the point that is closest to the query point. [Figure 12-9](#) shows a portion of the grid, the query point in red, and a circle drawn around the query point to show the distance to the nearest point

in that grid cell. It is clear from the figure that there is no need to search any further because no point outside that grid cell could possibly be any closer than the current closest point.

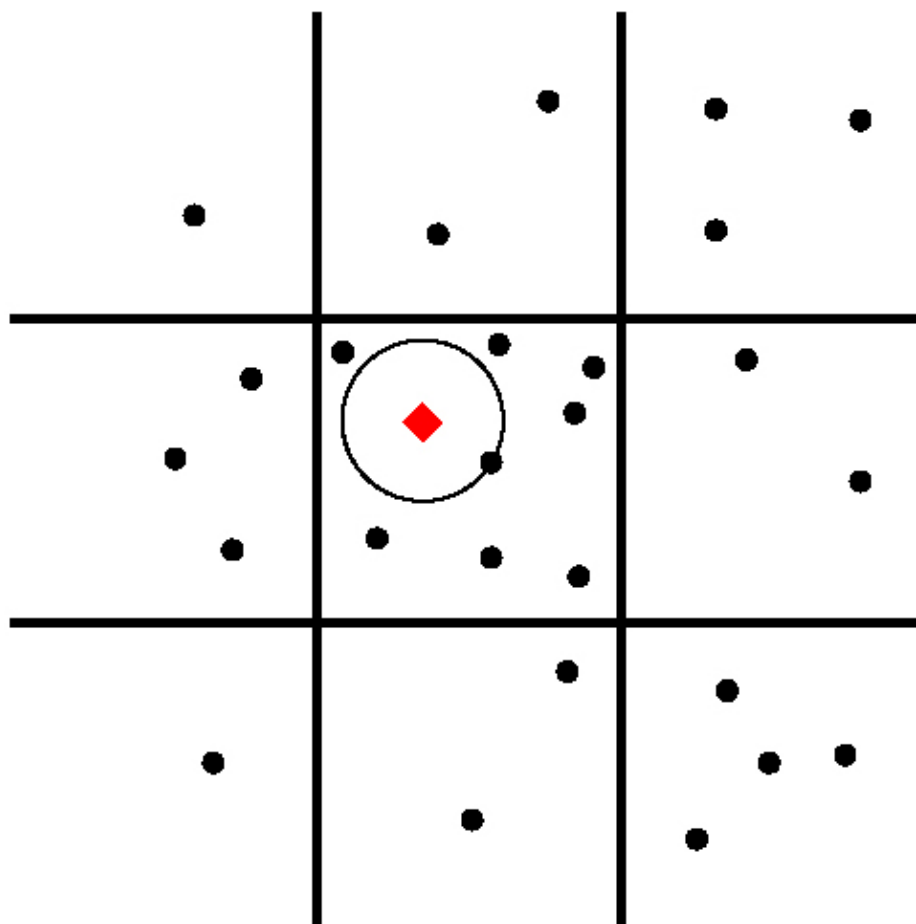


Figure 12-9 *Finding the nearest match in a grid*

Sometimes, though, you aren't so lucky. After checking the query point's grid cell, the closest point in that grid cell to the query point might be so far away that it is possible that a closer point could be in an adjacent grid cell. [Figure 12-10](#) illustrates this scenario. In this case, it is necessary to check the points in adjacent grid cells (shown shaded in light blue) to be sure that the closest point has been found.

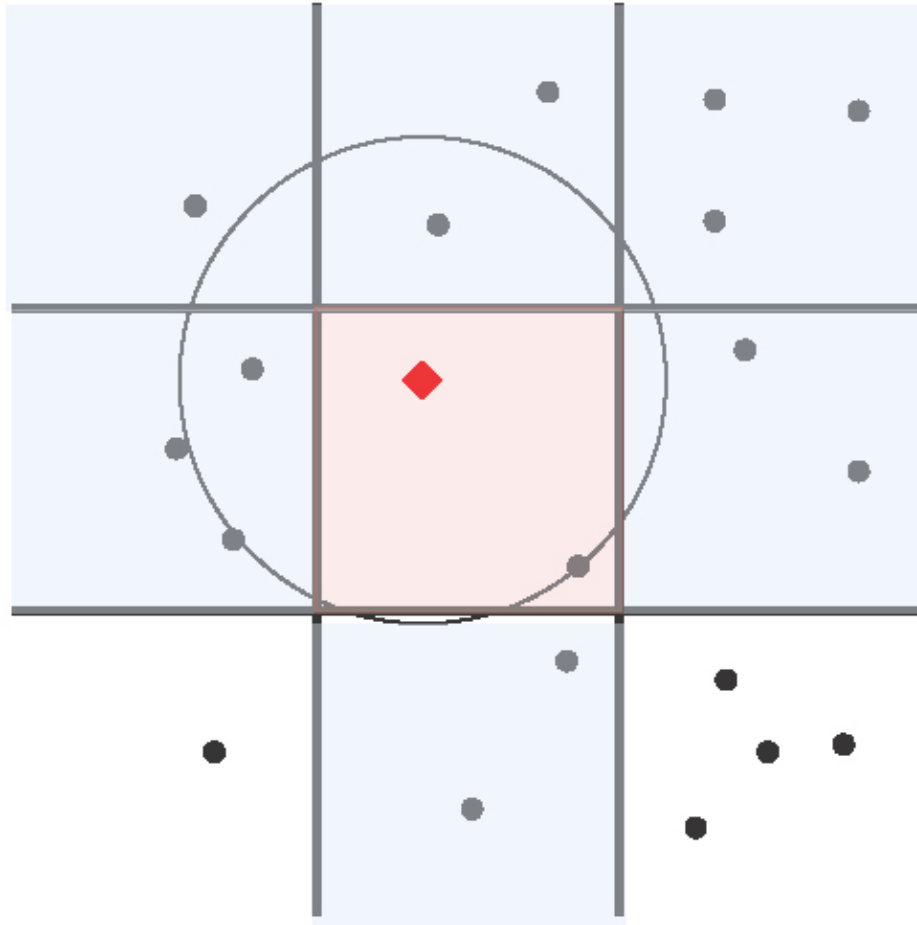


Figure 12-10 *Other grid cells to be checked to find the nearest point*

It is also entirely possible that there are no points at all in the query point's grid cell. In this case, it is necessary to check grid cells at distances farther and farther away from the query point until a point near (but possibly not nearest) the query point is found. This checking is done a *layer of cells* at a time, where each layer is one grid cell farther away from the query point's cell.

The search process can stop after the distance from the query point to the nearest point found so far is smaller than the distances from the query point to the edges of the entire layer whose points have all been compared to the query point.

Does the Query Circle Fall Within a Layer?

How can the algorithm be certain that `findNearest()` has found the point that is closest to the query point? After all the points within a layer have been

checked, and if the query circle is enclosed by that layer, then it is certain that there are no points in farther layers that could possibly be closer to the query point. In the case of [Figure 12-9](#), you know that there is no need to consider additional layers of cells. In the cases of [Figure 12-10](#) and [Figure 12-11](#), however, the query circle extends beyond the bounds of the current layer of cells, so it is necessary to consider the points in the next layer out. In [Figure 12-11](#), the cell containing the query point is empty, forcing the search to at least include the first layer. Stopping the search at the first layer, however, would miss potentially closer points in the second layer.

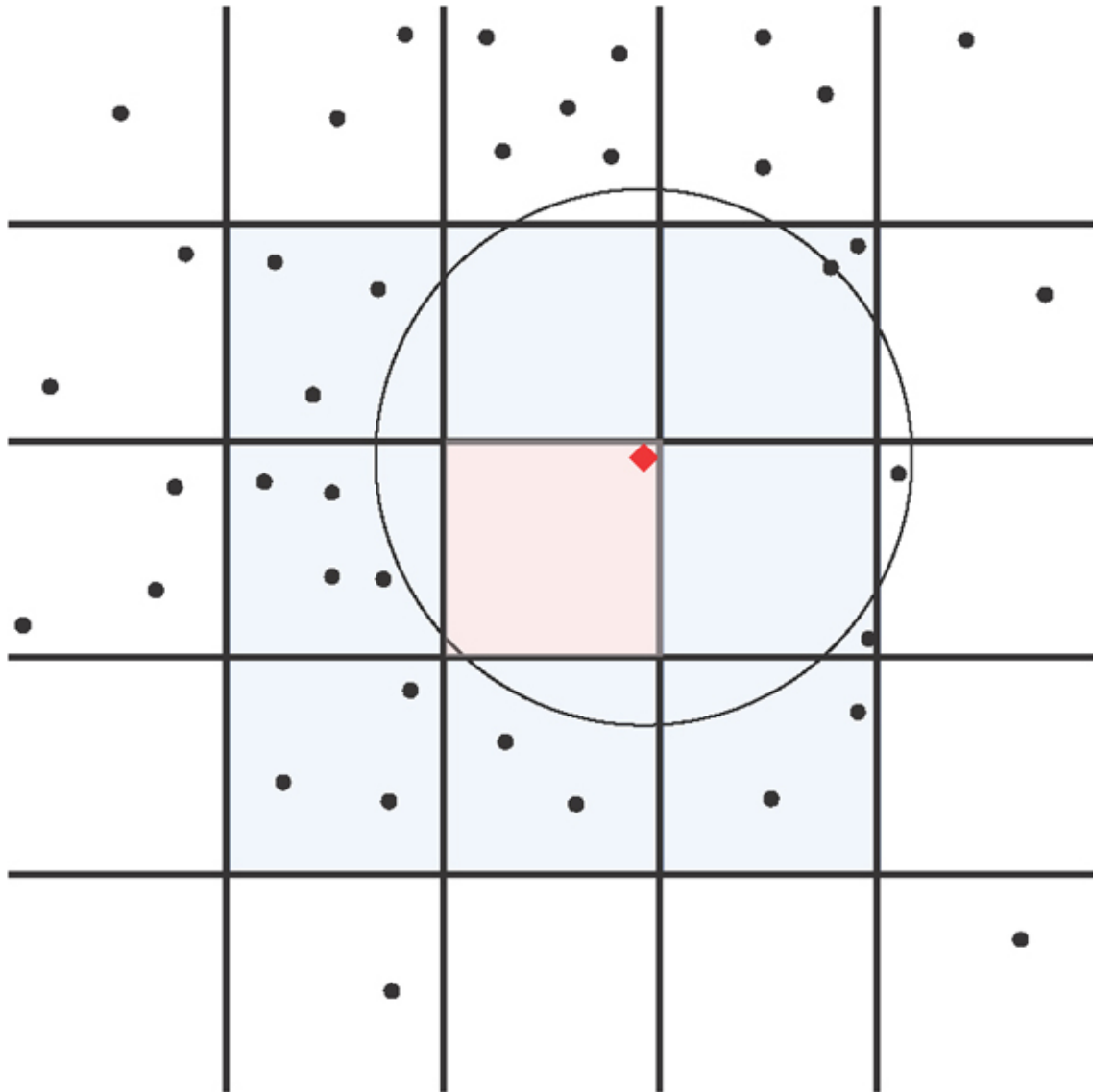


Figure 12-11 *Checking additional layers to find the nearest point*

The row and column number of a cell can be turned back into the coordinates of its lower-left corner by multiplying by the private `__cellSize` attribute. So, given the row and column of the cell containing the query point, the `__cellSize`, and the number of the layer that you're searching (assuming that the center cell containing the query point is at layer zero), it is easy to compute the left, right, top, and bottom bounds of the entire layer in either Cartesian or geographic coordinates.

For example, see [Figure 12-12](#), where the row, col of the center query cell is at 1, 2; the current layer is 2; and the cell size is 50 units in Cartesian coordinates. Because you're at layer 2, the entire square layer encompasses five rows and five columns of cells, each of which is 50×50 in Cartesian coordinates. Then the row of the bottom of the layer is $1 - 2$ or -1 , the row of the top layer is $1 + 2$ or 3, and in terms of Cartesian coordinates

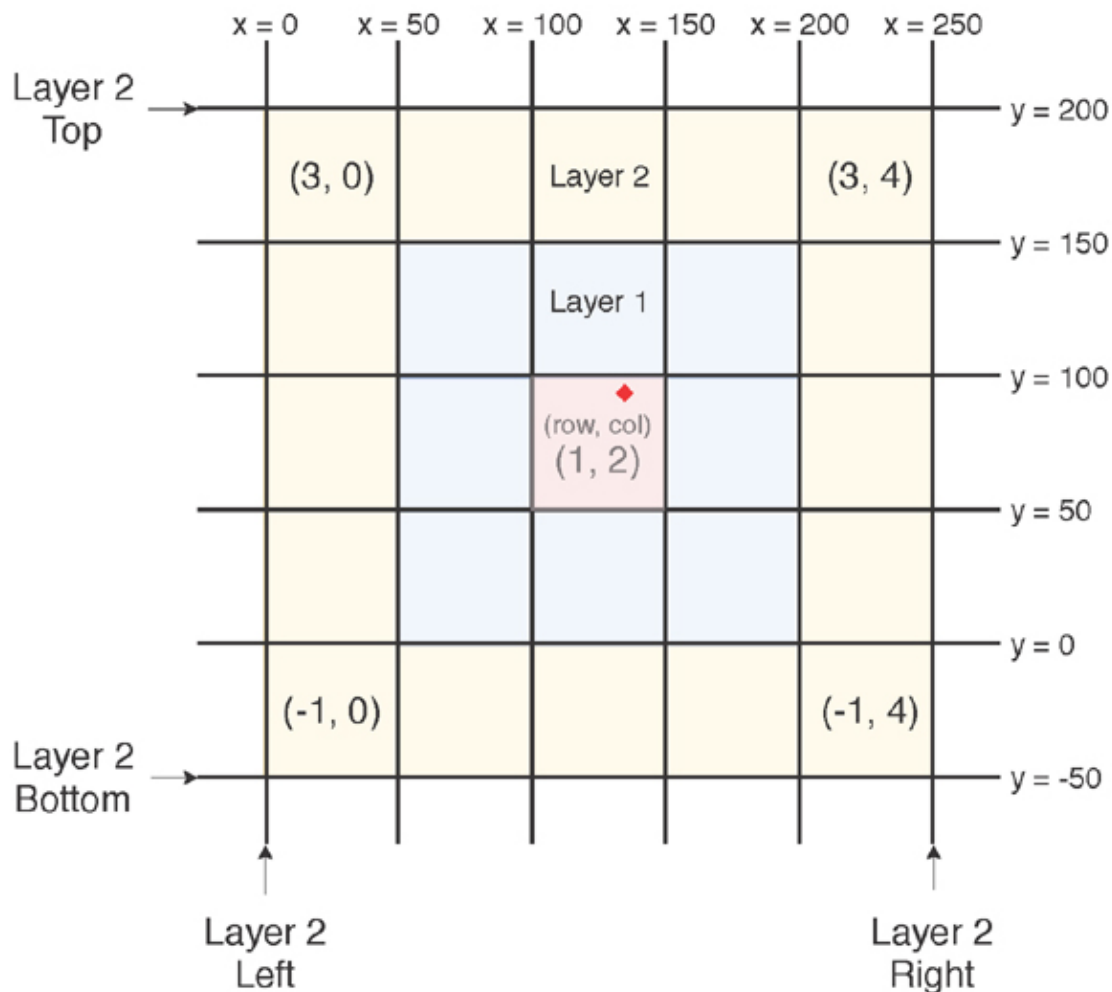


Figure 12-12 *Determining the boundary coordinates of a layer*

bottom edge: $y = (-1 * 50) = -50$ top edge: $y = ((3 + 1) * 50) = 200$

Likewise, the left and right coordinates of the current layer can be calculated from the column number of the query cell.

The `__getLayerBounds()` method in [Listing 12-18](#) takes a row, column tuple specifying the `cell`, and a layer number as parameters, and does this computation, returning a `Bounds` object containing the Cartesian or geographic coordinates of the left, right, top, and bottom edges of the current layer of cells.

Listing 12-18 *Computing the Bounds of a Layer*

```
class Grid(object):
...
    # return Bounds of the layer with cell at its center
    def __getLayerBounds(self, cell, layer):
        left = (cell[1]-layer) * self.__cellSize
        right = left + (self.__cellSize * (layer * 2 + 1))

        bottom = (cell[0]-layer) * self.__cellSize
        top = bottom + (self.__cellSize * (layer * 2 + 1))

        return Bounds(left, right, top, bottom)
```

It's now a simple matter to compute whether a query circle is entirely contained within the bounds of the rectangle centered at the query point and that extends out the specified number of layers, as shown in [Listing 12-19](#). First, you compute the location of the layer's outer boundaries, and then you use the `Bounds.within()` method (refer to [Listing 12-7](#)) to determine whether the query circle's bounding box falls within the bounds of the current layer.

Listing 12-19 *Determining Whether a Query Circle Lies Within a Layer*

```
class Grid(object):
...
    # Returns true if the query circle falls within the specified
    # layer surrounding the search coordinates.
    # a,b are the coordinates of the query circle center
    # cbounds is the current Bounds of the query circle
    def __withinLayer(self, cBounds, a, b, layer):
        # get the bounds of the layer
        cell = self.__getCell(a, b)          # row,col of cell
        layerBounds = self.__getLayerBounds(cell, layer)
```



```
# check if the circle's bounds are within the layer
return cBounds.within(layerBounds)
```

Does the Query Circle Intersect a Grid Cell?

The computation of distances between the query point and each point in a grid cell can be computationally expensive, especially for the computation of the haversine distance, which involves multiple trigonometric function calls. But you need to consider only the points contained in those cells that intersect with the query circle, and you can ignore the cells that don't intersect with the query circle.

[Figure 12-13](#) illustrates five query circle sample scenarios in which might occur when determining whether any part of a query circle falls within the center grid cell. By creating a bounding box surrounding the query circles, you can then easily determine whether a query circle might intersect with the center grid cell. Of course, approximating query circles with their bounding boxes has limitations. The approximation may sometimes falsely decide that a query circle intersects a grid cell, when in reality it is the corner of the bounding box that intersects with the grid cell, but not the circle itself. We have chosen to use bounding boxes of the query circle, even at the expense of occasional additional visits to a cell, to simplify the intersection computations for highly distorted query circles that are far from the equator.

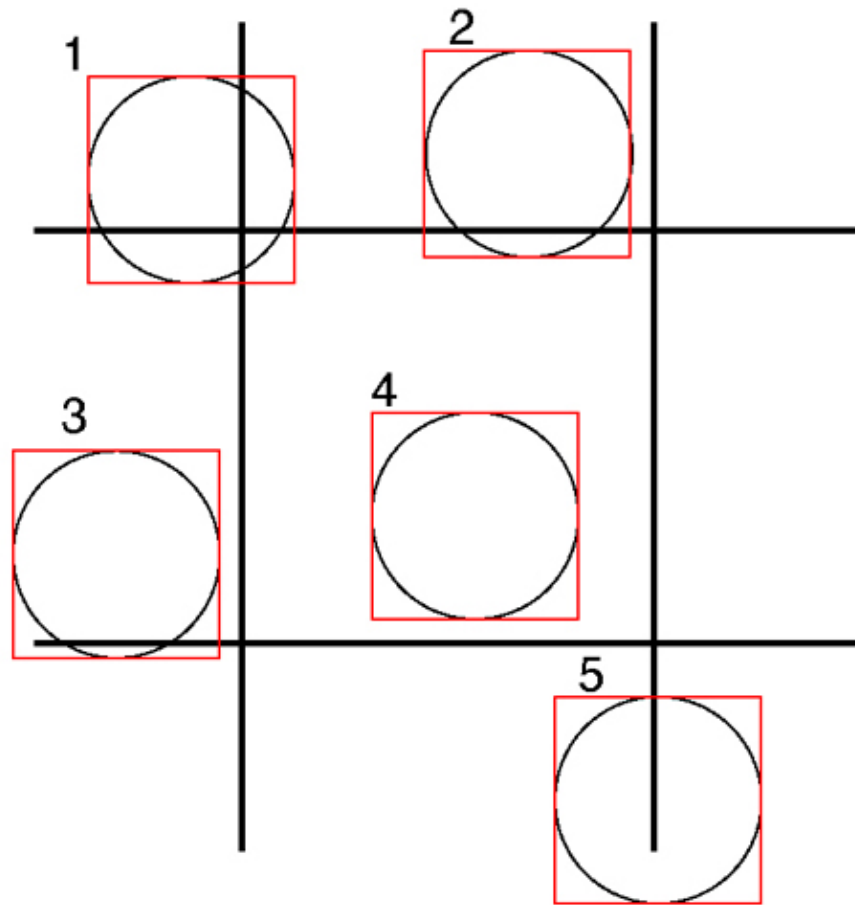


Figure 12-13 *Determining the intersection between a query circle and a cell*

If the circle's bounding box and cell don't intersect, then you can move on to the next cell. The `intersects()` method of the `Bounds` class in [Listing 12-6](#) performs this intersection test (with the bounding box approximation to the query circle).

Generating the Sequence of Neighboring Cells to Visit

The search for the point closest to the query point starts at the cell containing the query point and then works outward, considering the points in cells at successive layers of distance from the query point's cell. [Figure 12-14](#) shows the query point in red and then the neighboring cells labeled with the row and column offset from the query cell. The layer number can be easily derived from

the row, column offset by taking the maximum of the absolute values of the two offsets.

(2, -2)	(2, -1)	Layer 2 (2, 0)	(2, 1)	(2, 2)	
(1, -2)	(1, -1)	Layer 1 (1, 0)	(1, 1)	(1, 2)	
(0, -2)	(0, -1)	Layer 0 (0, 0) ♦	(0, 1)	(0, 2)	
(-1, -2)	(-1, -1)	(-1, 0)	(-1, 1)	(-1, 2)	
(-2, -2)	(-2, -1)	(-2, 0)	(-2, 1)	(-2, 2)	

Figure 12-14 *Offsets from the query center to each layer*

The `Grid` class contains a static method called `offsets()`, written as a generator, and shown in [Listing 12-20](#). The method yields tuples, one at a time, where each tuple is the row, column offset of the next grid cell to be visited. For each layer, `offsets()` first yields tuples for the cells along the sides of the layer, followed by the tuples for the corners of the layer. The corners are yielded last for each layer because the corner cells contain points that are potentially the farthest from the query point.

Listing 12-20 *A Generator to Produce Grid Offsets in Layer Order*

```
class Grid(object):
    # Generator to yield grid offsets in concentric
    # layers, starting at the center 0,0
    @staticmethod
    def offsets():
        yield 0,0

        layer = 1
        while True:
            for num in range(-layer+1, layer):
                yield num, layer # yield offsets for the
                yield num, -layer # cells along each side
                yield -layer, num
                yield layer, num

            yield -layer, layer # yield offsets for the
            yield layer, layer # corners of the layer
            yield layer, -layer
            yield -layer, -layer

            layer += 1 # move on to the next layer
```

This method could hypothetically run forever, yielding an infinite sequence of successively farther layers of grid cell offsets. As you soon see, however, `findNearest()` stops consuming offsets from the generator after the query circle fits within a layer. As a practical matter, if there are sufficient points per grid cell, it is rare that more than one layer of neighboring cells needs to be visited.

Pulling It All Together: Implementing Grid's `findNearest()`

With these preliminaries out of the way, you can finally implement `findNearest()`. This method is a bit longer than usual, so the code comments in [Listing 12-21](#) are annotated with lettered labels, which we expand on in the coming paragraphs.

A: If nothing has been inserted yet into the grid, there's no need to proceed further, so just return `None`. This is the only scenario in which

`None` is returned. Note that the `Grid` object does not keep track of the number of items it stores. The length of the Python dictionary (`__cells`) is the number of keys with values, so when there are no keys, no lists of points have been inserted.

B: The algorithm keeps track of the closest point seen so far, and the distance to that point. Initially, the closest point answer is `None`, and the distance to the closest point is `math.inf`. Because `inf` is the largest possible number, any subsequent point's distance will necessarily be closer.

C & D: The `__getCell` method converts the query point's coordinates into the row, column of the query point's cell. That cell is considered to be layer zero.

E: Loop over each of the grid cell offsets returned by the `offsets` generator. The first offset is (0,0) so that causes the search process to start at the cell containing the query point.

F: At some point, the next offset in the sequence will be one layer farther out. To determine when the layer number changes, we calculate the layer of this next offset. As seen in [Figure 12-14](#), the layer number is the larger of the absolute values of the row and column offset values.

G: If the next offset lies in a new layer of cells, this is the opportunity to check whether it is necessary at all to loop over the cells in the new layer. So, if an answer point has been found, and if the query circle fits entirely within the previous layer (`curLayer`), then there is no point in proceeding with the next layer. The method breaks out of the offset loop to return the best answer point so far. Otherwise, it records the next offset's layer as the current layer.

H: Having arrived at a new offset cell to search, `findNearest()` retrieves the list of points stored there. The cell's row and column are obtained by adding the offsets to the query point cell's row, column.

I: Now we're ready to decide whether to do the work to compare all the points in the new cell to the query point. Comparisons are only needed if there are some points in the cell, and if the query circle intersects the cell. Initially, when an answer hasn't yet been found, perhaps because all the grid cells so far have been empty, the distance is still infinity, so the query

circle will intersect every cell. But if the query circle doesn't intersect the cell, then none of the points could possibly be closer to the closest point encountered so far.

J: Looping through each of the points in the current cell, compute the distance from the point to the query point. If the distance is exactly zero, this means that search has found an exact match to the query point and can return the point and its data. Otherwise, if the distance to the query point is less than the smallest distance found so far, remember the new candidate point and its distance. Because the distance to the nearest point has now decreased, the method invokes `CircleBounds.adjust()` to recompute the bounding box for the smaller query circle.

K: There's only one way to make it to this line of code. Namely, that the `break` statement was executed during step **G** because an answer was found and because the query circle was entirely contained within the previous layer of searched cells. The `findNearest()` method returns the coordinates of the point, the data stored with that point, and the distance from the query point to the answer (that is, nearest) point.

This process may seem complex. Yet, as we show at the end of the chapter, in certain situations this `Grid.findNearest()` can perform significantly better than the theoretically superior `QuadTree` code.

Listing 12-21 *The `findNearest()` Method for `Grid`*

```
class Grid(object):
...
    def findNearest(self, a, b): # find the closest point to (a,b)
        if len(self.__cells) == 0: # A: Nothing in grid yet
            return None

        answer = None # B: remember the closest point so far

        # The current search circle and its bounds so far
        distance = inf
        cBounds = CircleBounds(a, b, distance, self.__distance)

        cell = self.__getCell(a, b) # C: cell containing a,b
        curLayer = 0                # D: the layer we're up to

        # E: for each offset
```

```

for off in Grid.offsets():
    # F: what layer is this new offset?
    layer = max(abs(off[0]), abs(off[1]))

    # G: if we're about to consider a new layer,
    # but the search circle falls entirely within
    # the prior layer, then there's no need to continue.
    if layer != curLayer and answer and \
        self.__withinLayer(cBounds, a, b, curLayer):
        break
    curLayer = layer # remember what layer we're up to

    # H: get the points stored in the cell at that offset
    offsetCell = cell[0]+off[0], cell[1]+off[1]
    points = self.__cells.get(offsetCell, None)

    # I: if there are points in the cell, and
    # the search circle intersects the cell...
    if points and \
        cBounds.intersects(self.__getCellBounds(offsetCell)):

        for p in points: # J: for each point in the grid cell
            # compute distance to that point from query point
            newDist = self.__distance(a, b, p[0], p[1])

            if newDist == 0: # exact match?
                return p[0], p[1], p[2], 0

            if newDist < distance: # new point closer?
                distance = newDist # remember the distance
                answer = p # and the point, and
                cBounds.adjust(distance) # adjust bounds

    # K: returns a, b, data, distance to query point
    return answer[0], answer[1], answer[2], distance

```

Quadtrees

In the previous sections, you saw that a grid is an excellent data structure for searching points when the data is somewhat uniformly spread out because each grid cell contains a relatively equal number of points for searching. But what happens if the data is not so uniformly spread out? [Figure 12-15](#) shows about 30,000 points. Each of the clusters has about one-third of the points. This kind of clustered data is typical in the real world. For example, locations of

restaurants, coffee shops, and other businesses tend to be clustered near each other.

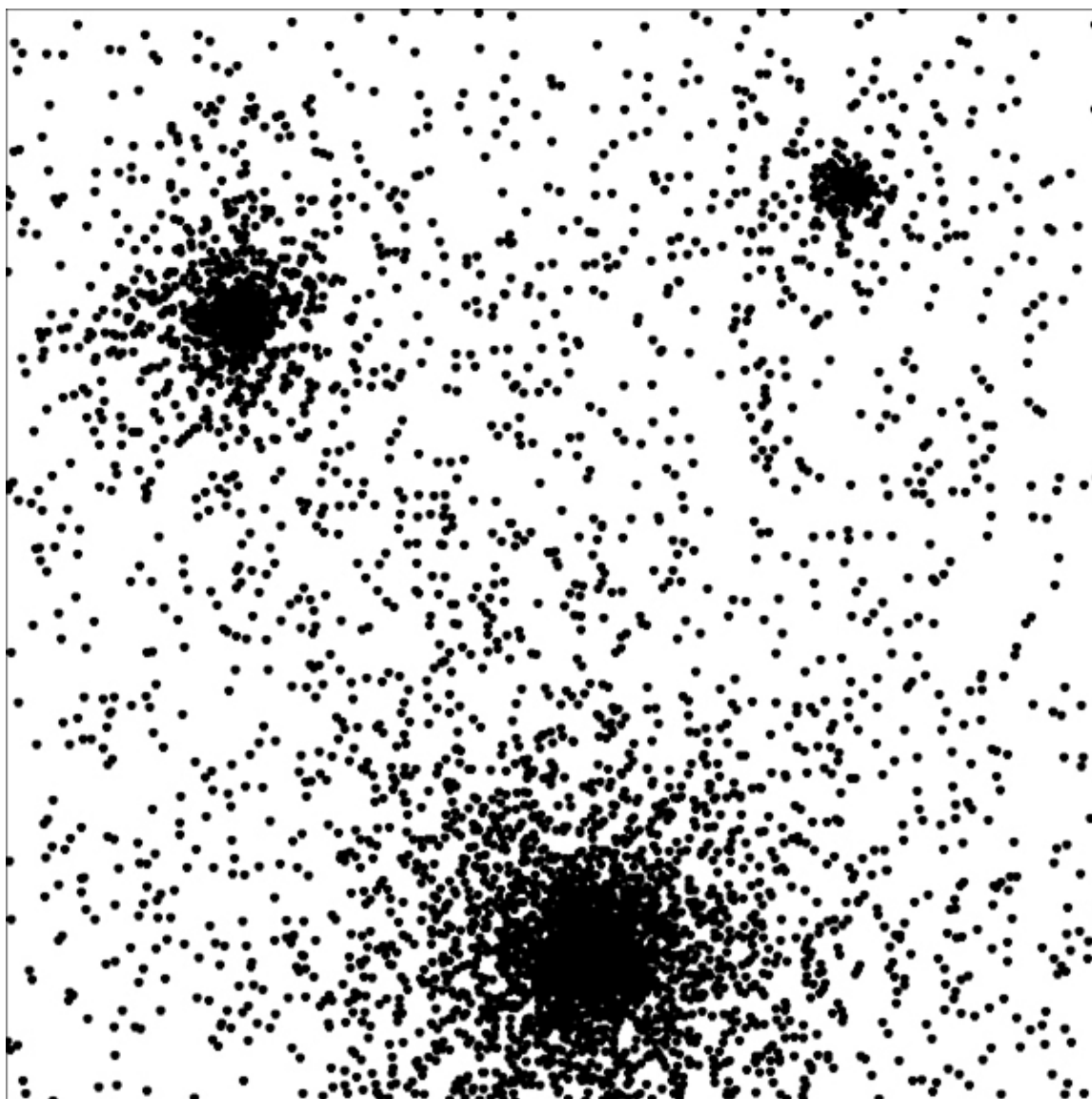


Figure 12-15 *Thirty thousand highly clustered points*

Overlaying a grid on top of this data, as shown in [Figure 12-16](#), makes it clear that for clustered data a grid doesn't offer much benefit. Searching for a point inside the red-outlined grid cell near the bottom would involve processing nearly all the points in that cluster.

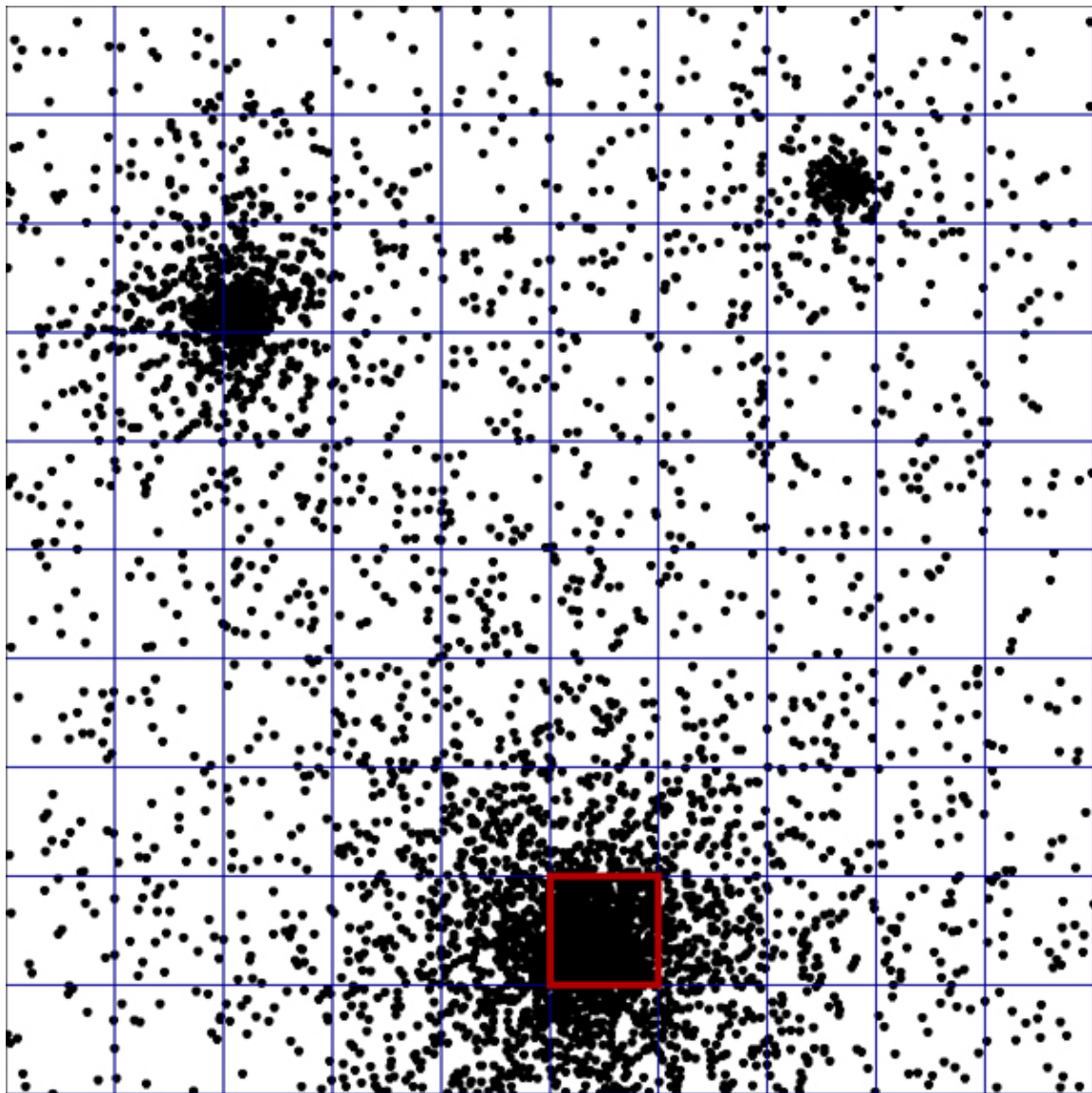


Figure 12-16 *Uniform grid dividing the 30,000 points*

Many of the grid cells are empty or contain just a few points, while most of the points are concentrated in a small number of the grid cells. As one solution, you could decrease the size of every one of the grid cells. So instead of a grid of 10×10 cells overall, you'd use a grid of 100×100 cells for a total of 10,000 grid cells to store the 30,000 points. [Figure 12-17](#) zooms in to look at just one of the 10×10 cells, the red-outlined one, which contains several thousand points.

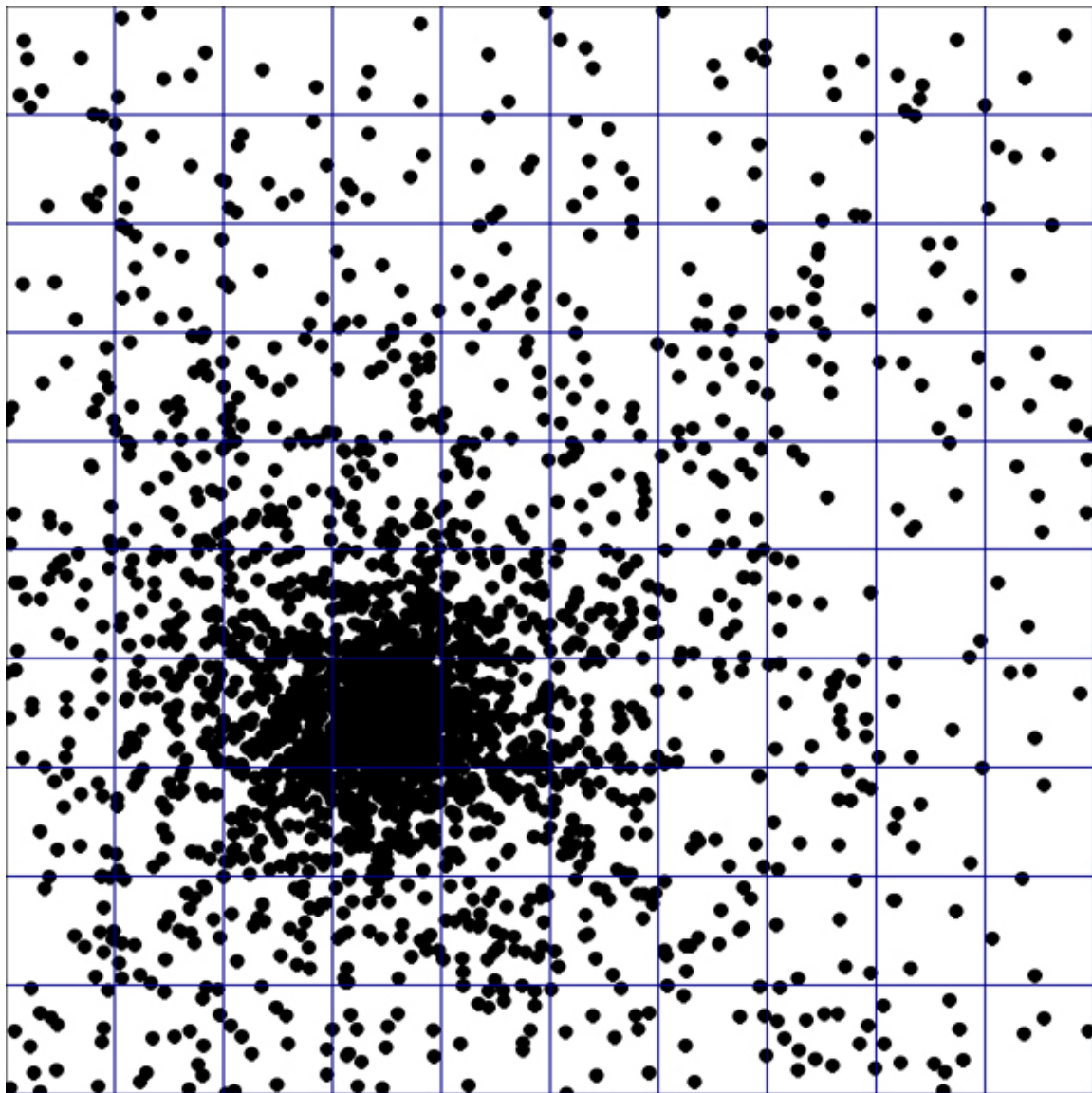


Figure 12-17 *A finer grid on just one of the point clusters*

At this scale, because you are now using a 100×100 grid, what used to be a single grid cell is now itself composed of 10×10 of the smaller cells. And again, most of the points are heavily concentrated within a small fraction of the cells, while most of the cells contain a relatively small number of points.

This is the dilemma with using fixed grids where the cell size is uniform across all the data. Real-life data tends to be very unevenly distributed. At whatever scale of magnification that you use to examine the data, clusters tend to be apparent. The implication is that no matter how fine the grid is made, some areas of space would benefit from a finer grid.

Quadtrees, also written as *quad trees*, provide a solution to this problem. A **quadtree** is a tree data structure that recursively divides up space into four quadrants. Parts of the quadtree that contain more points are further divided into finer-sized quadrants. Parts of the quadtree that contain fewer points divide space into larger quadrants. Adaptively subdividing space based on the density of spatial data makes it possible to insert or search for points in $O(\log_4 N)$ time, or as you saw in [Chapter 2](#), “[Arrays](#),” more simply just $O(\log N)$ time.

This tree-like structure probably looks familiar. The different kinds of trees you studied in [Chapters 8, 9, and 10](#) were all organized by dividing the items by their keys. Quadtrees do something similar but with two numeric values. A binary search tree with a single numeric key inserts items with lower-valued keys in the left subtree and higher-valued keys in the right subtree. With spatial data, you can do something similar, but this time with quadrants.

In the following pages, we look in more detail at a quadtree implementation using the same methods that were implemented for the `PointList` and `Grid` classes: a constructor, `insert()`, `findExact()`, `findNearest()`, `delete()`, and `traverse()`.

Creating an Instance of the `QuadTree` Class

Client code creates an instance of the `QuadTree` class by invoking the constructor to create an object that contains Cartesian points:

```
q = QuadTree('cartesian')
```

or that contains geographic points:

```
q = QuadTree('geographic')
```

If the constructor is passed any string other than `'cartesian'` or `'geographic'`, an exception is raised.

[Listing 12-22](#) shows the code for the `QuadTree` constructor. It has just two private attributes: a reference to the distance function appropriate to either Cartesian or geographic coordinates, and an initially empty reference to the root node of the tree.

Listing 12-22 *The `QuadTree` Constructor*

```
class QuadTree(object):

    def __init__(self, kind): # must specify type of coordinate
        if kind == 'cartesian':
            self.__distance = euclideanDistance
        elif kind == 'geographic':
            self.__distance = haversineDistance
        else:
            raise Exception("Invalid kind of coordinate")

        self.__root = None
```

Inserting Points: A Conceptual Overview

Let's look at the first few inserts of points into an initially empty quadtree, with each new point finding its place in the quadtree through a recursive descent. Each node in a quadtree potentially has four children, corresponding to the northeast, southeast, southwest, and northwest quadrants of space that surround that point. The first point to be inserted into the quadtree becomes the root of the tree (labeled point A in [Figure 12-18](#)), and the four children of that node all refer to `None`.

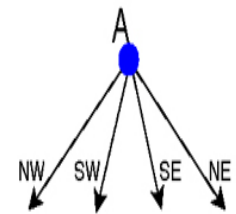
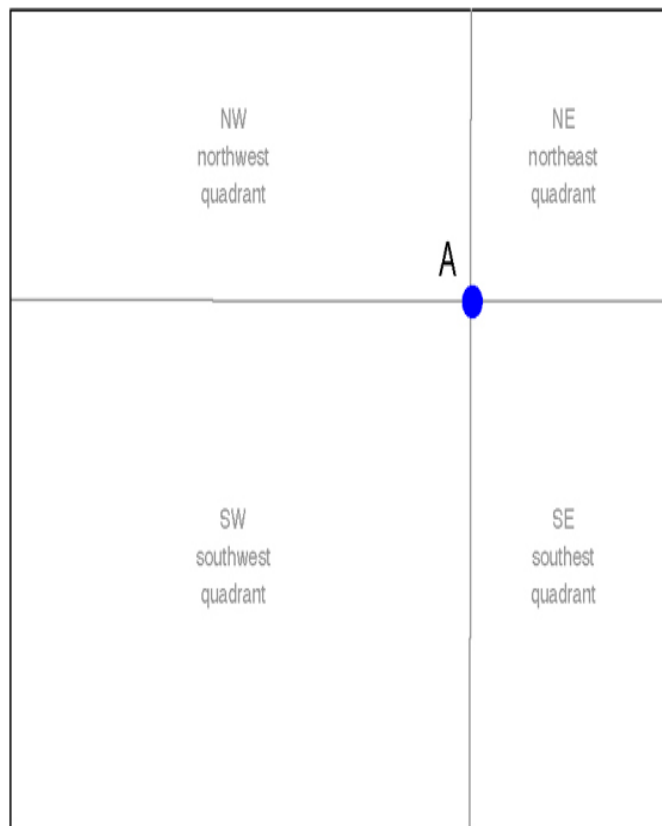


Figure 12-18 *Inserting the root point in a `QuadTree`*

Next, as shown in [Figure 12-19](#), another point, labeled B, is inserted. Starting at the root, the coordinates of the new point are compared against the coordinates of the root node A. Point B lies to the southwest of point A, so you recursively descend to the corresponding child of point A. Because the southwest child currently refers to `None`, a new node is created to contain B, and upon return from the recursion, the southwest child of A is made to refer to B.

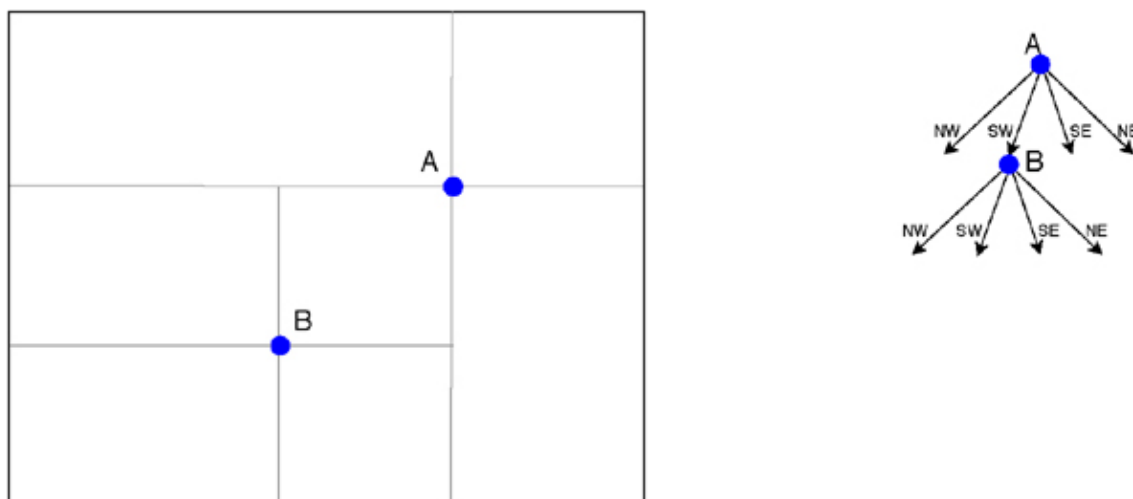


Figure 12-19 *Inserting point B*

The next point, labeled C is inserted, as shown in [Figure 12-20](#). Again, starting from the root node, C is compared to the coordinates of point A. Point C lies to the southeast of A, so the recursion proceeds to the southeast child, and because it is found to be `None`, a new node is created containing the coordinates of point C, which is then returned from the recursion and assigned to the southeast child of A.

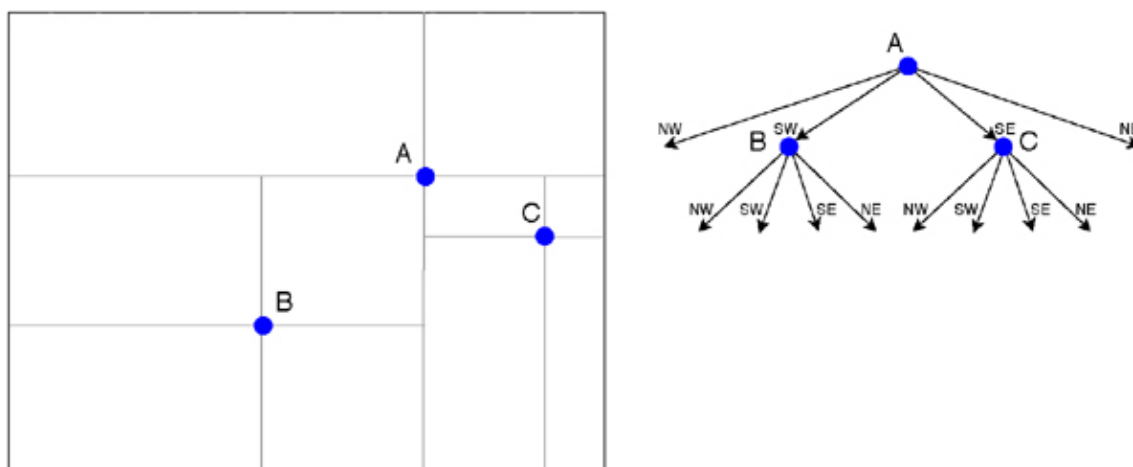


Figure 12-20 *Inserting point C*

Finally, as shown in [Figure 12-21](#), a point, labeled D, is recursively inserted into the tree, starting at the root point A. Because D lies in the southwest quadrant of A, the recursion then proceeds to point B, which is the southwest child of point A. The coordinates of point D are northwest of the coordinates of point B, so the recursion then proceeds to the northwest child of B. Having

reached `None`, a new node is created to hold point D, which is then returned from the recursion. Upon returning to point B, the northwest child of B is set to refer to the just-returned reference to the newly created point D.

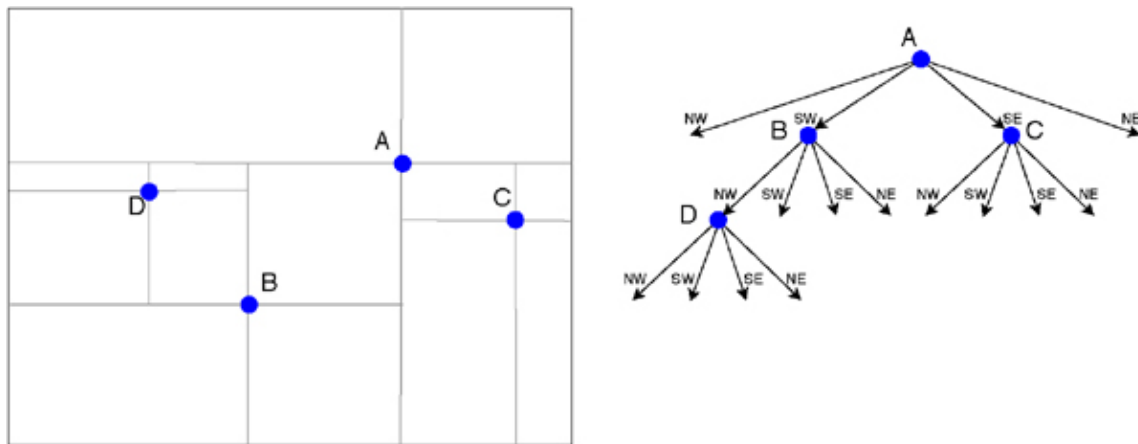


Figure 12-21 *Inserting point D*

Avoiding Ambiguity

When recursively descending a quadtree either for insertion or searching, it is important to make a consistent decision at each node regarding which child to descend into. If a point is properly within a quadrant, then there is no ambiguity. But which quadrant should be chosen if an inserted or searched point lies directly above or below, or directly to the right or left of a node? For example, in [Figure 12-22](#), to which quadrant of A does point B belong?

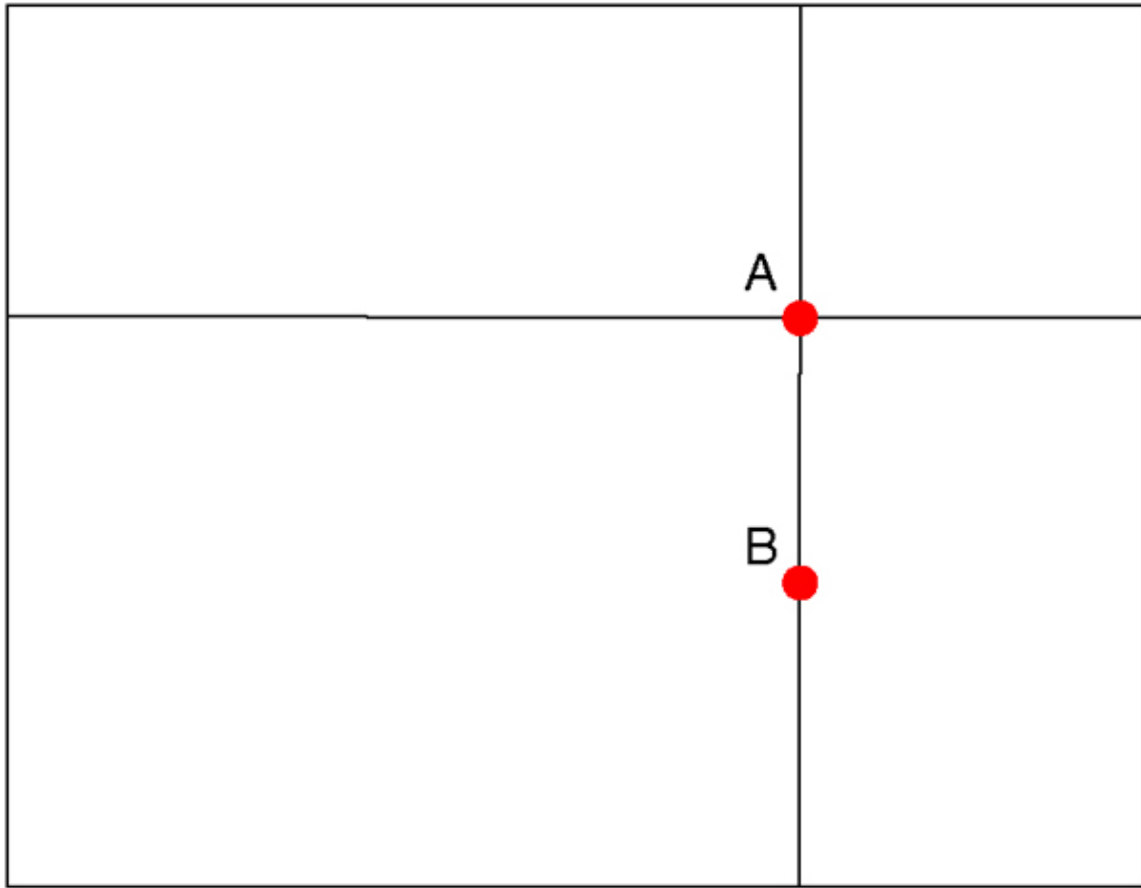


Figure 12-22 *Quadrant ambiguity*

Whatever solution we choose, we must make certain to apply it consistently. Our solution to this problem is to have the points on each of the four axis boundary lines belong to the quadrant that is clockwise of the axis line. So, in [Figure 12-22](#), B would belong to the southwest quadrant of A.

More generally, as shown in [Figure 12-23](#), the points that are exactly above the node's point all belong to the node's northeast quadrant. The points that are exactly to the right of the quadtree node belong to the node's southeast quadrant, and so on.

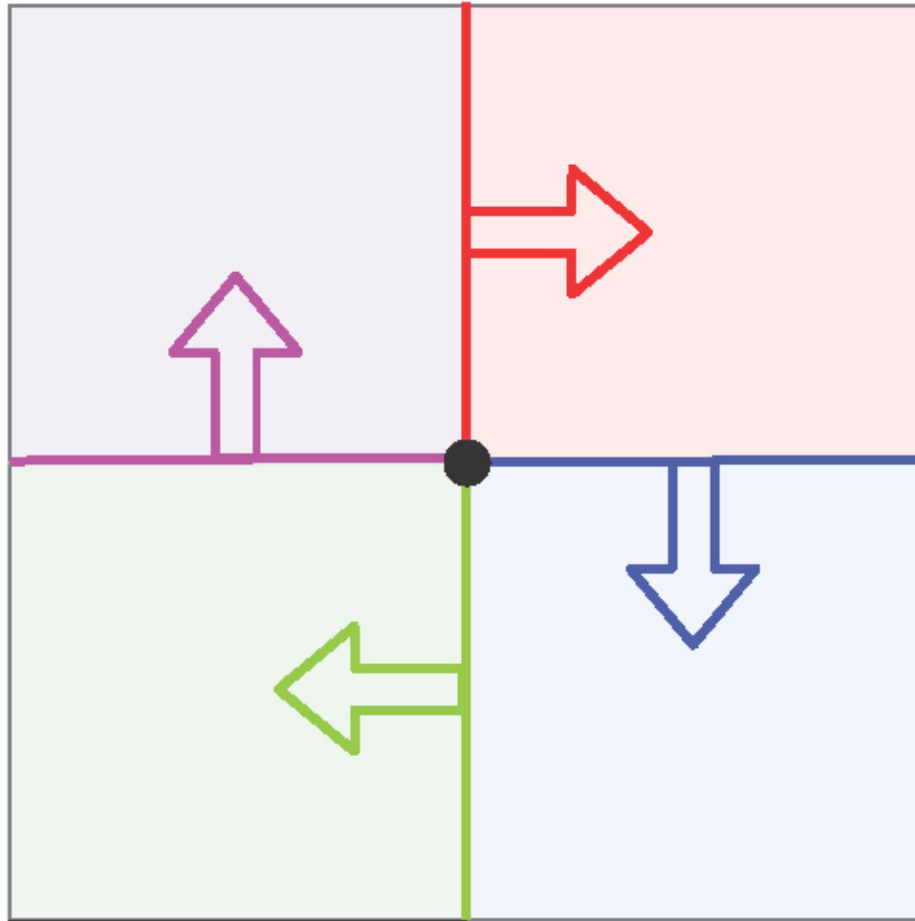


Figure 12-23 *Assignment of axis boundary lines to quadrants*

The QuadTree Visualization Tool

To see how quadtrees operate, you can run the QuadTree Visualization tool. The tool can be used to manage Cartesian point data in a rectangular region, as shown in [Figure 12-24](#). The points are shown as dots with a label above them that represents the “data” associated with a point.

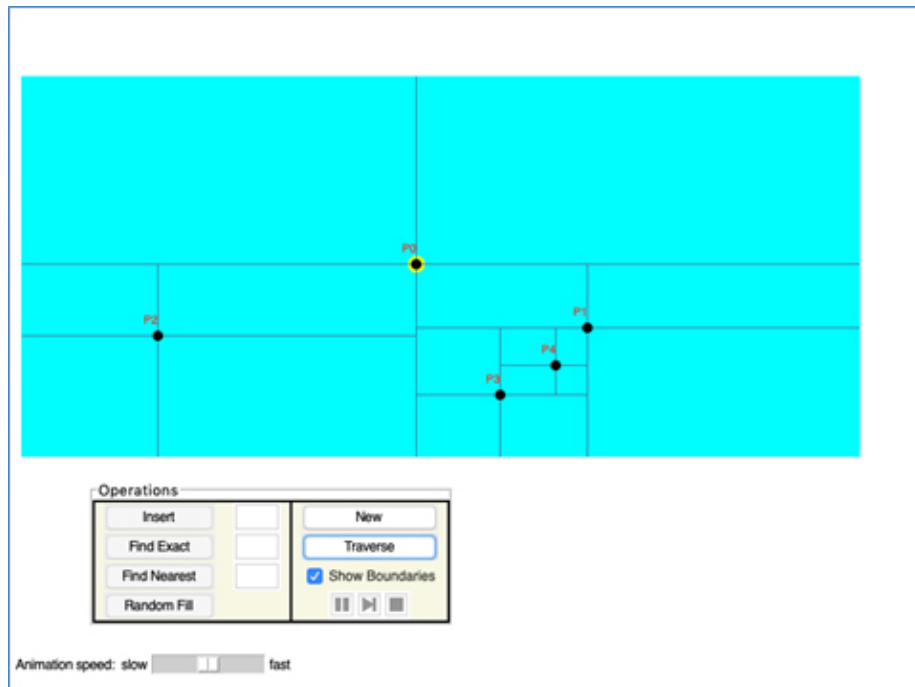


Figure 12-24 *The QuadTree Visualization tool*

You can insert new points in two ways: by double-clicking with a pointer (mouse) button in the light blue region, or by entering an X coordinate, Y coordinate, and data label in the three text boxes and selecting the Insert button.

The first point inserted becomes the root of the quadtree and is highlighted in yellow. When you insert points by double-clicking, the tool automatically chooses the data label in the form P0, P1, P2, and so on, based on the current number of points in the quadtree. When you type the coordinates, they are relative to the origin, point (0, 0), at the lower left of the region. You can provide any short string for the data.

The quadrants around each point are visible when the boundaries between them are displayed. If you uncheck the Show Boundaries checkbox, you will see only the points, making their tree relationship invisible. Finding the parent of a node is somewhat tricky. You must look at the boundaries, follow them to their endpoint, and then follow the crossing boundary to see whether it intersects a labeled node. For example, in [Figure 12-24](#), P3's parent is P1, and two of P3's boundary lines end on the boundary lines of P1. The parent of P4 is harder to see because both P1 and P3 look equally likely based on the closest boundary lines. You must look at the extent of the boundaries of the two potential parents to resolve the ambiguity. P0, the root, has boundary lines that extend all the

way to the blue region boundaries, and those have no boundary lines leading to another node.

Try inserting a few points by typing the coordinates for the points. By giving exact coordinates, you can insert new points on a quadrant boundary. Based on the rule in [Figure 12-23](#), you should be able to predict the quadrant of the new point. The (single) boundary line drawn for it spans that quadrant. The other boundary line overlaps the boundary line where the point lies, so it is not visible. You can also click an existing point to copy its coordinates and label to the text input boxes. From there, you can edit the values to insert a new point that lands exactly on one of its quadrant boundaries. Clicking the blue region where there are no points simply enters the two coordinates into the text boxes.

Implementing Quadtrees: The `Node` Class

Just as with other types of trees, you can define a class to correspond to the nodes within a quadtree, as shown in [Listing 12-23](#). The `Node` class and its attributes don't have to be made private because the `QuadTree` class will never directly return an object of the `Node` type, and therefore, encapsulation will not be violated.

Listing 12-23 *The Node Class*

```
class Node(object):
    # a,b corresponds to either x,y or long,lat
    def __init__(self, a, b, data):
        self.a = a # coordinates of the point
        self.b = b
        self.data = data # data associated with point

    # Four children of the Node
    self.NE = self.SE = self.SW = self.NW = None
```

Having seen the constructors you may wonder, where in the `Node` or `QuadTree` classes are the boundary lines for each quadrant stored? The answer is that the boundary lines are not explicitly stored anywhere! Rather, you can think about the *a*, *b* coordinates of each `Node` as establishing the boundaries that subdivide all of the space surrounding the point into four quadrants. For example, the southeast quadrant of a `Node` has its top boundary specified by the *b* coordinate, and its left boundary specified by its *a* coordinate. But what about the bottom

and right boundaries? Those boundaries are not explicitly stored in the `Node` objects; rather, they are implicit. That is partially the reason why it is difficult to see the parent-child relationships from the boundaries shown in the visualization tool—the boundaries of a child extend to its parent’s boundaries.

As you will see shortly, the implementation of `findNearest()` search recursively descends the quadtree, progressively refining the boundaries of the space of the quadrant being visited. For example, the boundaries of the root `Node` at (a, b) are, respectively:

left: `-math.inf`, right: `math.inf`, top: `math.inf`, bottom: `-math.inf`

In other words, the root node’s space encompasses all points in the plane. Suppose you descend to the southeast corner of the root node, `P0`, which would be node `P1` in [Figure 12-24](#). Then you can refine the boundaries for the southeast quadrant of `P0` at (a, b) to be

left: `a`, right: `math.inf`, top: `b`, bottom: `-math.inf`

In other words, the southeast quadrant of the root is thought of as being unbound to the right and downward. As you descend farther into the tree, the left, right, top, and bottom boundaries get updated to reflect the subset of the plane that is associated with a child `Node`. In many parts of the implementation, however, these boundaries don’t need to be explicitly stored. For example, they are not needed for insertion. We discuss the explicit boundaries in more detail when we discuss `findNearest()`.

The insert Method

`QuadTree` clients insert point data using an `insert()` method with the same signature as those in `PointList` and `Grid`. [Listing 12-24](#) shows the Python code for `insert()` and its companion method, `__insert()`.

Listing 12-24 *The insert() Method of QuadTree*

```
class QuadTree(object):
...
    def insert(self, a, b, data): # Wrapper. Always succeeds.
        self.__root = self.__insert(self.__root, a, b, data)

    # Recursive private method that does the inserting
    def __insert(self, n, a, b, data):
```

```

# return a new Node if we've reached None
if not n: return Node(a, b, data)

# if the point to be inserted is identical to current node,
# overwrite the data, and don't recurse any further
if n.a == a and n.b == b:
    n.data = data
    return n

# recurse down into the appropriate quadrant
if a >= n.a and b > n.b: n.NE=self.__insert(n.NE,a,b,data)
elif a > n.a and b <= n.b: n.SE=self.__insert(n.SE,a,b,data)
elif a <= n.a and b < n.b: n.SW=self.__insert(n.SW,a,b,data)
else:
    n.NW=self.__insert(n.NW,a,b,data)
    # a < n.a and b >= n.b

return n

```

The `insert()` method is a wrapper method whose sole function is to invoke the recursive `__insert()` method, passing to it the root of the quadtree. Whatever `__insert()` returns is then assigned back to the `__root` attribute. This unusual pattern is worthy of some discussion, as you may recall from when it was used in [Chapter 10](#), “AVL and Red-Black Trees,” for `AVLTree` objects.

Why should `insert()` need to reassign a value for `__root` upon every invocation? Consider the case when the quadtree does not yet contain any points, and therefore `__root` refers to `None`. When `insert()` invokes `__insert()`, a new `Node` object is created and then returned immediately to `insert()`, which then assigns it to `__root`. That makes sense! But what about subsequent invocations of `insert()`? Would its recursive call to `__insert()` ever return a reference to a different `Node`? No, it would not. But that is OK, because the assignment to `self.__root` just ends up assigning the same `Node` to it that is already there! As you look more carefully at `__insert()`, you'll see that you can ask the same question regarding the returns from all the layers of recursion.

Moving on to `__insert()`, the method takes as parameters a reference to the current `Node n` in the `QuadTree`, along with the *a*, *b* coordinates and the associated data for the point to be inserted. In the first step, if `__insert()` reaches an empty child node (`None`), then it creates a new `Node` and returns it. The caller might have been the `insert()` wrapper, in which case the new `Node` will be assigned to `self.__root`. The caller could also have been one of the

recursive invocations of `__insert()` later in the method. When those invocations return, the returned newly created `Node` gets assigned into the quadrant attribute that until now referred to `None`.

If `n` refers to anything other than `None`, then the next check is for the possibility that the newly inserted point `a, b` and the current node `n` both refer to the exact same coordinates. We made a design decision that `QuadTree` objects will not support multiple points with the duplicate coordinates. So, if there is an exact match, then the newly inserted data will overwrite the existing data stored for those coordinates, and then `__insert()` can return a reference to the current node `n`. We leave it as an exercise to explore how to insert points with duplicate coordinates.

Finally, the real work of the recursion happens. At this point of the `__insert()` method, `a, b` can't possibly be an exact match to `n`'s coordinates. The `a, b` coordinates of the inserted point are compared to the coordinates of the current node `n` to decide which quadrant should receive the new point. Let's take a careful look at the first of these four similar recursion decisions:

```
if a >= n.a and b > n.b: n.NE = self.__insert(n.NE, a, b, data)
```

Notice that points will end up in the northeast quadrant if the `a` coordinate is greater than or equal to the current `Node`'s `a` coordinate and if the `b` coordinate is greater than the current `Node`'s `b` coordinate. But that is exactly the decision that we illustrated in [Figure 12-23](#): ambiguous points are placed in the quadrant clockwise away from the boundary where they lie. The decisions for recursion into the other three quadrants are likewise consistent with the figure.

Also notice the peculiar form of the recursive invocation. It recursively passes `n.NE` into `__insert()`, and then whatever the invocation of `__insert` returns gets assigned right back into `n.NE`! This is just a variation of the first invocation of `__insert()` by the `insert()` wrapper method. Here, when `__insert()` is recursively invoked, `n.NE` might be `None`, or it might already refer to a child `Node`. If `n.NE` is `None`, then immediately upon the recursive invocation, `__insert()` will create and return a new `Node` containing the inserted `a, b` data point. The caller appropriately assigns the new node to `n.NE`. If `n.NE` is not `None` and already refers to a child `Node`, then `__insert()` will necessarily end up returning that very `Node`, which then gets harmlessly assigned back to `n.NE`.

Appropriately, the last line of `__insert()` causes a reference to the current `Node n` to be returned to the calling method.

In summary, there are three possible returns from the recursion:

1. A reference to a brand-new `Node` is returned if the recursion reached an empty node represented as `None`.
2. If a duplicate point was inserted, after overwriting the current node's data with the newly inserted data, a reference to the current `Node` is returned.
3. A reference to the current `Node` `n` is returned if the insertion recursively occurred somewhere below the current `Node`.

In all three cases, what is returned to the calling method is a reference to the `Node` that belongs at that point in the `QuadTree`. Most of the time, the returned value references a `Node` that has already been created and that is already referred to by one of the child attributes of the parent `Node`.

Another way to think about this is to imagine the process involved in inserting a point with new coordinates 10 levels down into the `QuadTree`. You recurse down 10 levels into the appropriate quadrants and reach a child attribute containing `None`. At that point, you create a new `Node`, and it is returned to be assigned to the parent's appropriate child attribute. As the recursion then "unwinds" all the way back to the root, each intermediate `Node` is just returned to the calling method, which then harmlessly assigns the `Node` back to the child attribute that already contains that reference.

Try stepping through the insertion of a point using the visualization tool. It shows how each recursive call to `__insert()` checks the relationship of the new point's coordinates with those of a node in the tree and then either descends to a child node, creates a new node, or updates the data of an existing one. As the recursion unwinds, the nodes visited along the path to the new or updated node are returned to their parent.

Efficiency of Insertion

If the quadtree is relatively well balanced, then inserts into the quadtree should take $O(\log N)$ time, so inserting N points would take $O(N \times \log N)$ time. Just as with binary search trees, degenerate sequences of inserts can cause the tree to be very unbalanced. For example, if points were inserted in order along a southwest diagonal line, the result would be the worst possible case, with each insertion taking $O(N)$ time, resulting in $O(N^2)$ time to insert N points. You see

shortly that such an unbalanced quadtree degrades the performance of searching, too.

Dynamic rebalancing of a `QuadTree` in a manner like an AVL tree is possible but very complex. If an application has all, or even most, of the points in advance that need to be stored in a `QuadTree`, then a bulk insert method can arrange for the tree to be better balanced. We leave this method for you to complete as an exercise at the end of the chapter.

Finding an Exact Match

The client uses the `findExact()` method to search for the point contained in the `QuadTree` object that exactly matches the specified *x*, *y* or longitude, latitude coordinates. If an exact match is found, the method returns a reference to the data stored for the matching point. The client uses `findExact()` in the same fashion as with the `PointList` and `Grid` classes.

Finding an exact match is a simple process, as shown in [Listing 12-25](#). If the `QuadTree` contains an exact match, then a recursive descent that mirrors the descent used by `insert()` is guaranteed to find the matching point.

Listing 12-25 *The `findExact()` Method of `QuadTree`*

```
class QuadTree(object):
...
    # Wrapper method - returns the data object associated with a,b
    def findExact(self, a, b): return self.__find(self.__root, a, b)

    def __find(self, n, a, b):
        if not n: return None # Couldn't find the exact point

        # Did we find the exact point? Return the data
        if n.a == a and n.b == b: return n.data

        # Recurse down into the appropriate quadrant.
        # If a,b is anywhere, it MUST be in that quadrant.
        if a >= n.a and b > n.b: return self.__find(n.NE, a, b)
        elif a > n.a and b <= n.b: return self.__find(n.SE, a, b)
        elif a <= n.a and b < n.b: return self.__find(n.SW, a, b)
        else:
            return self.__find(n.NW, a, b)
            # a < n.a and b >= n.b
```

Like the `insert()` method, you can use a wrapper method called `findExact()` to start the recursion by passing the root node into the recursive `__find()` method and simply returning its result. The first base case occurs if the recursion has reached a `None` (empty leaf) value. If this happens, it is certain that there's no exact match for a, b and `None` should be returned to indicate that no point was found, and therefore no data returned.

Another base case occurs if the search has reached a node that exactly matches a, b . In this case, the associated data is returned and makes its way back to the wrapper method as the recursion unwinds.

Otherwise, if an exact match to a, b is anywhere in the quadtree, it must be in the quadrant that encloses a, b . So, just as was done for insertion, you recursively descend into the appropriate quadrant, and you return whatever is returned by the recursive call. If the recursion reaches `None`, then it is certain that there's no node that matches the specified coordinate.

Try a few exact searches with the visualization tool to watch this process. Create a few points in the quadtree and then click a region with no points. Doing so fills in the coordinates of the clicked point in the arguments area and enables the Find Exact operation. Select the Find Exact button. It draws a cross at the clicked point and makes the recursive calls to `__find()` as it checks the relationship of a, b to each node. You can also click an existing point to verify that `findExact()` follows the nodes in the tree to that point.

Efficiency of Exact Search

If the quadtree is relatively well balanced, then finding an exact match for a point in a quadtree should take $O(\log N)$ time, though in the worst case with a completely unbalanced tree, `findExact()` could be as bad as $O(N)$. This degenerate performance makes it no better than the operation in a simple `PointList`, although it is very unlikely to occur with data that is inserted in a random order with respect to spatial position.

Traversing the Points

Just as we did with `PointList` and `Grid`, `QuadTree`'s `traverse()` implementation should be done as a generator. The natural way to traverse a `QuadTree` is to do a pre-order recursion, starting at the root, and then recursively visiting and yielding each of the nodes in the tree, followed by each

node's children. Mixing recursion with generators poses a challenge because it is difficult to write efficient recursive generators in Python.

As we discussed in [Chapter 6, “Recursion,”](#) every recursive function can be transformed into a nonrecursive version by using a stack. We showed this transformation in [Chapter 8, “Binary Trees.”](#) [Listing 12-26](#) shows a similar nonrecursive implementation of `traverse()`, where we use a Python `list` to implement the two required stack operations of push (using `list`'s `append()` and `pop()` methods). Alternately, it would be straightforward to use the `Stack` class from [Chapter 4, “Stacks and Queues,”](#) if you know the number of points in the quadtree, or the `LinkStack` class from [Chapter 5, “Linked Lists,”](#) if you don't.

Listing 12-26 *The `traverse()` Method of `QuadTree`*

```
class QuadTree(object):
...
    def traverse(self):
        s = [ ]      # initialize stack with root
        if self.__root: s.append(self.__root)

        while len(s) > 0: # stack's not empty?

            # process Node at top of stack
            n = s.pop()
            yield n.a, n.b, n.data

            # push each child onto stack
            if n.NE: s.append(n.NE)
            if n.SE: s.append(n.SE)
            if n.SW: s.append(n.SW)
            if n.NW: s.append(n.NW)
```

The method starts by initializing the stack to contain just the root `Node`. Then, as long as the stack isn't empty, you pop the top `Node` from the stack and yield its coordinates and data. Finally, any children of the `Node` are pushed onto the stack.

This implementation seems more complex than the corresponding `traverse()` methods in the `PointList` and `Grid` classes. What is its Big O complexity?

With a bit of analysis, we can determine that the quadtree's `traverse` is also $O(N)$. We determine this by focusing on the pushes and pops of points to and

from the stack. Examining the code, we can see that each `Node` is pushed just once onto the stack, and that each `Node` is ultimately popped (and yielded) just once from the stack, corresponding to each pass through the `while` loop. Thus, for a `QuadTree` containing N nodes, the traversal can be accomplished with N passes through the `while` loop, and a total of N pushes and N pops. These roughly $3N$ operations correspond to $O(N)$ complexity for the `traverse()` method.

Deleting a Point

To locate a point to delete, you should use the same approach as used in `findExact()`. If the matching point is identified, it needs to be deleted. If not, an exception should be raised.

Removing a leaf node from a quadtree should be relatively simple. What should the method do, however, if the matching node is an interior node of the quadtree? Because a `Node` can't be easily removed from the quadtree without rebuilding the structure of its children, one reasonable approach would be to leave the `Node` in place, and instead set its data attribute to `None`. Thus, any `Node` with a data attribute containing `None` would not subsequently be treated as a valid data point stored in the quadtree. If the quadtree must support `None` as a legitimate data value for a point, then a special sentinel value must be inserted that can be distinguished from `None`. The `__Deleted` sentinel performs this role for open addressing hash tables (as described in [Chapter 11](#), “[Hash Tables](#)”).

Marking the data attribute as deleted means that small changes are needed to the `findExact()` method (in [Listing 12-25](#)) and the `findNearest()` method (described in the following sections) so that nodes without data are never considered as valid matching points. These data-less nodes take some extra memory but still provide useful information about where space is subdivided into quadrants.

Because `delete()` must first find the node like `findExact()`, the Big O must be at least $O(\log N)$. The rest of the work (removing a leaf node or inserting a no-data marker in the data attribute of an interior node) takes a few steps but does not depend on the number of items in the quadtree, so it adds nothing to the overall complexity. If many interior nodes are deleted, the tree does not shrink, so the N in $O(\log N)$ is the total number of inserted items, not the current number of items in the tree, for deletion, insertion, and search.

We leave it as a Programming Project for you to implement a method to delete a point specified by a pair of coordinates from the quadtree. The visualization tool does not have a delete operation, but you can erase all points with the New button and then insert new points.

Finding the Nearest Match

As usual, finding the nearest match via the `findNearest()` method is more complicated than `findExact()`. Just as with the `Grid` class, merely descending into the appropriate quadtree quadrant does not guarantee that you have found the nearest match to the search coordinates.

For example, in [Figure 12-25](#), if you descend into the `QuadTree` to find the deepest node whose boundaries surround the red diamond query point (outlined in red), you can use that deepest point as a good *candidate* for the nearest match. However, if you draw a circle with the query point at its center and its radius matching the distance to the candidate, it becomes clear that any quadrant that is overlapped by the circle *might* conceivably contain a closer node and must therefore be considered. Those quadrants that you must visit are shaded in light blue. The example shows that there is a point in the query circle that was not visited on the path to the candidate (because the search from the root node—the one whose quadrant boundaries span the entire figure—followed the path to the NW, then NE, and then SW quadrants based on the query point). How do you ensure that those other quadrants shaded in blue are visited during the search?

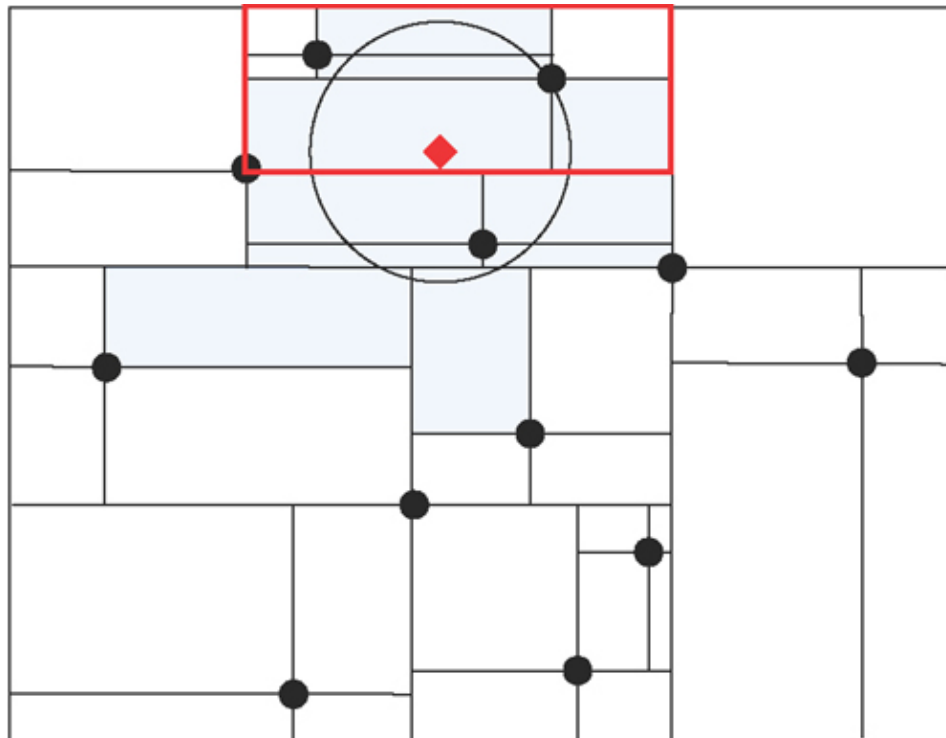


Figure 12-25 *Additional quadrants to be visited based on query circle*

Similar to how the `Grid` object had to intersect a query circle with the various cells, there's a way to compare quadtree boundaries with the query circle to minimize the quadrants to explore. Let's look at the implementation details to complete your understanding of how to find the nearest `Node` to a query point.

Finding a Candidate Node

Let's start by following the logic used in `findExact()` to get a point somewhere in the vicinity of the query point. [Listing 12-27](#) shows the private `__nearPoint` method. As you soon see, `findNearest()` invokes this method by passing in `__root` and the *a*, *b* search coordinates.

Listing 12-27 *The `__nearPoint()` Method*

```
class QuadTree(object):
...
    # find a nearby (but not necessarily the nearest) point
    # to a,b by recursing as deep as possible into the tree.
    def __nearPoint(self, n, a, b):
        # Base cases:
```

```

if not n: return None # reached None so return it

if a == n.a and b == n.b: return n # found exact point

# recurse down into the appropriate quadrant
if a >= n.a and b > n.b: ans = self.__nearPoint(n.NE, a, b)
elif a > n.a and b <= n.b: ans = self.__nearPoint(n.SE, a, b)
elif a <= n.a and b < n.b: ans = self.__nearPoint(n.SW, a, b)
else:
    ans = self.__nearPoint(n.NW, a, b)
    # a < n.a and b >= n.b

# if we found a lower Node near this point return it
# otherwise return the current node
return ans if ans else n

```

The `__nearPoint()` method returns the deepest `Node` in the tree along the path from the root toward the query point. It does so by recursively descending into the `QuadTree` in a fashion almost identical to `__find()`. Instead of returning the corresponding data if an exact match is found to a, b , `__nearPoint()` returns a reference to the `Node` that either matches a, b exactly, or is as deep as possible in the `QuadTree`.

Not surprisingly, the base cases are similar to `__find()`. If `__nearPoint()` reaches `None`, then `None` is returned. If it reaches a node whose coordinates exactly match a, b , then that `Node` (n) is returned.

Otherwise, it recursively descends into the quadrant containing the query point and stores in `ans` whatever is returned by the recursive invocation. There are only two possibilities after the recursive call. One possibility is that `ans` is `None` because `__nearPoint()` just descended into an empty child. If that happens, then it means that the current node n is a leaf, and therefore it returns the current node, n . Otherwise, if `ans` is anything but `None`, it means that the recursion is returning a reference to the lowest possible `Node`, which is then returned to its caller.

Finding the Closest Node

After `__nearPoint()` has discovered a candidate for the `Node` nearest to a, b , the `__nearest()` method shown in [Listing 12-28](#) can recursively descend the quadtree again, starting at the root, trying to find successively closer candidates. This method computes the distance from the query point to the coordinates of each `Node` visited. If the distance is smaller than the distance to

the so-far best candidate Node, then the current node, *n*, becomes the new, closer candidate, *cand*.

Listing 12-28 *Recursively Refine Nearest Candidate*

```
class QuadTree(object):
...
    # Returns the nearest Node and distance to query point a,b.
    def __nearest(self, n, # current Node being visit
                  a, b,   # query point
                  dist,   # distance to candidate
                  cand,   # best candidate so far
                  bounds): # Bounds of current quadrant

        # Reached a None node, or already found
        # a perfect match? Return our answer.
        if not n or dist == 0: return cand, dist

        # Is the current quad tree point closer than
        # the candidate? If so, update the candidate
        fn = self.__distance
        newDist = fn(a, b, n.a, n.b)
        if newDist < dist:
            cand = n
            dist = newDist

        # bounding box surrounding the search circle
        cBounds = CircleBounds(a, b, dist, fn)

        # For each child node - update bounds for that quadrant
        # if the search circle's bounds intersects the child
        # quadrant's new bounds, descend to that child node.
        newB = bounds.adjusted(left = n.a, top = n.b)
        if cBounds.intersects(newB):
            cand, dist = self.__nearest(n.SE, a, b, dist, cand, newB)
            cBounds.adjust(dist) # adjust the circle's bounds

        # likewise for the other three quadrants
        newB = bounds.adjusted(left = n.a, bottom = n.b)
        if cBounds.intersects(newB):
            cand, dist = self.__nearest(n.NE, a, b, dist, cand, newB)
            cBounds.adjust(dist)

        newB = bounds.adjusted(right = n.a, top = n.b)
        if cBounds.intersects(newB):
            cand, dist = self.__nearest(n.SW, a, b, dist, cand, newB)
```

```

        cBounds.adjust(dist)

    newB = bounds.adjusted(right = n.a, bottom = n.b)
    if cBounds.intersects(newB):
        cand, dist = self.__nearest(n.NW, a, b, dist, cand, newB)
        # no need to update circle's bounds after last quadrant

    return cand, dist # best candidate seen so far

```

In the `Grid` class, you saw how the `intersects()` method determines whether the query circle's bounds intersect a grid cell and, therefore, whether to consider the points within the cell. `QuadTree`'s `__nearest()` method also uses the same `intersects()` method to determine whether the query circle's bounds intersect any of the quadrants below the current node and, therefore, whether to continue the recursion into that quadrant. To make that determination, `__nearest()` must manage `Bounds` objects for both the query circle and the quadrant associated with node `n`. The `bounds` parameter passed from the parent holds node `n`'s boundaries, and the node `cBounds` variable is set to be the query circle's bounds using the new nearest distance to `n`. As the algorithm descends into each node's quadrants, these bounds will be updated, as described shortly.

Each return from the four possible recursive calls to `__nearest()` returns either the same candidate and distance that were just passed to the `__nearest()` recursion or a closer candidate and distance that the recursion discovered. Because it is possible that a recursive call may have decreased the distance to the nearest candidate, `__nearest` also invokes `CircleBound`'s `adjust()` to adjust the bounding box around the current query circle. Having possibly checked all four quadrants, `__nearest()` then returns the best candidate and distance that was found.

There is some subtlety in how `__nearest()` uses `Bounds.intersects()`. As previously discussed, `Node` objects do not explicitly store the specific boundaries of the rectangle of space that they cover. Rather, each node stores just two of the boundaries, defined as appropriate, by the *a* coordinate (for either the left or right boundary) and by the *b* coordinate for either the top or bottom boundary.

How do you determine all four boundaries that define the rectangle surrounding each node? Consider [Figure 12-26](#), which shows a `QuadTree` containing just three nodes: a root, `G`, drawn in green; its northeast child, `P`, drawn in purple; and `P`'s southeast child `C`, drawn in (cobalt) blue. The nodes form a grandparent